Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Debugging pada Level Instruksi

Pertemuan 14

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 4.1 dan 4.2

Sub-CPMK 4.1 — Melakukan penelusuran program pada level instruksi (*single-step*, *trace*, *breakpoint*) dan memantau perubahan register serta *flag*.

Sub-CPMK 4.2 — Memeriksa dan menganalisis isi memori dan *stack* (*dump*) untuk menemukan kesalahan logika, kesalahan segmentasi, dan *overflow*.

CPMK yang didukung: CPMK04 ▪ **CPL yang didukung: CPL03**

Setelah pertemuan ini, mahasiswa mampu:

1. membedakan lima jenis kesalahan program: sintaks, saat berjalan, logika, segmentasi, dan *overflow*;
2. memilih alat *debugging* yang sesuai: MS-DEBUG, *debugger* GUI Turbo Assembler, atau Emu8086;
3. memakai perintah `R`, `U`, `T`, `P`, `G`, `D`, `E`, `A`, `N`, `L`, dan `W` pada DEBUG;
4. menjelaskan perbedaan `T` dan `P` ketika menjumpai `CALL` dan `INT`;
5. mengisi tabel *trace* register dan *flag* setiap langkah eksekusi;
6. membaca keluaran `D` serta menafsirkan bingkai *stack*;
7. menerapkan metodologi *debugging* sistematis untuk menemukan tiga bug khas.

Peta Materi

1. **Taksonomi Kesalahan Program dan Alat Debugging**
   lima jenis kesalahan; gejala dan cara penemuannya; MS-DEBUG, GUI Turbo Assembler, dan Emu8086

2. **Menelusuri Program dengan Perintah DEBUG**
   `N`, `L`, `W`, `R`, `U`, `T`, `P`, `G`, `D`, `E`, `A`; perbedaan `T` dan `P`; *breakpoint*; tabel *trace*

3. **Analisis Memori dan Stack serta Metodologi Debugging**
   membaca *dump*; bingkai *stack*; deteksi *overflow*; enam tahap metodologi; tiga studi kasus

Praktik pada pertemuan ini

Menelusuri `bug-loop.asm` dan `bug-stack.asm` dengan MS-DEBUG, mengisi tabel *trace*, lalu memperbaiki kedua program.

BAGIAN

Taksonomi Kesalahan Program dan Alat Debugging

Debugging sebagai Proses Sistematis

Menulis program assembly berarti berhadapan langsung dengan register, *flag*, dan alamat memori tanpa lapisan pelindung apa pun.

Definisi

Debugging adalah proses sistematis untuk menemukan penyebab suatu kesalahan (*bug*) pada program, membuktikan penyebab itu melalui bukti pengamatan, lalu memperbaiki dan memverifikasi hasil perbaikan.

- *Debugging* bukan kegiatan menebak-nebak, melainkan kegiatan mengumpulkan bukti pada level instruksi.
- Bekal utamanya adalah tiga pengetahuan dari bab sebelumnya: cara kerja register, pengambilan keputusan berdasarkan *flag*, dan pemakaian *stack* oleh prosedur.

Sumber: Bab XV, Taksonomi Kesalahan Program dan Alat Debugging

Lima Jenis Kesalahan Program

1. **Kesalahan sintaks** — penulisan yang membuat assembler menolak berkas sumber, sehingga program *tidak* dihasilkan (mis. `MOVV` alih-alih `MOV`).
2. **Kesalahan saat berjalan** — baru muncul ketika program dijalankan, misalnya pembagian dengan nol.
3. **Kesalahan logika** — program berjalan sampai selesai tetapi hasilnya salah; assembler dan sistem operasi sama-sama diam.
4. **Kesalahan segmentasi** — program mengakses alamat di luar segmen yang seharusnya, misalnya `DS` tidak pernah diarahkan ke segmen data.
5. **Overflow** — hasil perhitungan atau pemakaian *stack* melebihi daya tampungnya, misalnya melebihi `FFFFh` atau jumlah `PUSH` tidak seimbang dengan `POP`.

Sumber: Bab XV, Lima Jenis Kesalahan Program

Gejala dan Cara Menemukan Tiap Jenis

Jenis	Gejala	Kapan terdeteksi	Cara menemukan
Sintaks	Assembler menolak berkas; tidak ada `.obj`	Sebelum program dijalankan	Baca pesan assembler beserta nomor barisnya
Saat berjalan	Program berhenti di tengah jalan; pesan sistem operasi	Ketika dijalankan	Jalankan bertahap sampai titik berhenti
Logika	Program selesai normal, hasil salah	Tidak terdeteksi otomatis	Tabel *trace* register dan *flag* per langkah
Segmentasi	Karakter sampah, nilai berubah sendiri, program macet	Ketika dijalankan	Periksa `DS`, `ES`, `SS`, dan batas indeks melalui `D`
Overflow	Hasil aritmatika terpotong; `RET` melompat ke alamat aneh	Ketika dijalankan	Periksa `CF`, `OF`, dan pasangan *dump stack*

Penting: Hanya kesalahan sintaks yang dapat dipercayakan kepada assembler. Empat jenis lainnya menuntut pengamatan langsung terhadap keadaan prosesor saat program berjalan.

Sumber: Bab XV, Tabel taksonomi kesalahan

Tiga Alat Debugging yang Tersedia

Ketiganya mampu melakukan hal yang sama: menghentikan program, memeriksa register, dan memeriksa memori. Perbedaannya terletak pada kemudahan pemakaian dan kedekatan dengan mesin sesungguhnya.

1. **MS-DEBUG di DOSBox.** Program bawaan DOS yang seluruh interaksinya berupa baris perintah. Yang terlihat benar-benar isi memori, bukan tampilan yang disederhanakan.
2. ***Debugger* terpadu GUI Turbo Assembler.** Satu jendela terpadu memuat kode sumber, register, dan memori sekaligus; program dijalankan baris per baris dengan menekan tombol.
3. **Jendela *debug* Emu8086.** Register, *flag*, memori, dan *stack* tampil sebagai panel terpisah yang selalu terlihat sehingga cocok untuk pemula.

Mengapa MS-DEBUG ditekankan

DEBUG memperlihatkan keadaan mesin apa adanya, dan kemampuan membaca keluarannya berlaku umum bagi kedua alat grafis lainnya.

Sumber: Bab XV, Alat Debugging yang Tersedia

Perbandingan Kemampuan Tiga Alat

Kemampuan	MS-DEBUG (DOSBox)	GUI Turbo Assembler	Emu8086
Tampilan	Teks, baris perintah	Grafis, jendela terpadu	Grafis, panel terpisah
Langkah per instruksi	`T` dan `P`	Tombol `Step`	Tombol `Single Step`
Breakpoint	`G` pada alamat; jumlah terbatas	Titik henti pada baris sumber	Titik henti pada baris sumber
Memeriksa memori	`D` dan `E`	Jendela *dump* memori	Jendela memori
Memeriksa *stack*	`D SS:SP`	Jendela *stack*	Jendela *stack*
Memodifikasi kode mesin	`A` dan `E`	Tidak langsung	Tidak langsung
Menjalankan `.exe` lama	Ya	Ya	Perlu emulasi

Sumber: Bab XV, Tabel perbandingan alat debugging

Rangkuman Bagian 1

- *Debugging* adalah proses sistematis yang membuktikan penyebab kesalahan melalui bukti pengamatan pada level instruksi — bukan kegiatan menebak-nebak.
- Lima jenis kesalahan adalah sintaks, saat berjalan, logika, segmentasi, dan *overflow* (aritmatika maupun *stack*).
- Hanya kesalahan sintaks yang dapat ditemukan tanpa *debugger*; kesalahan logika paling sulit karena program selesai normal tetapi hasilnya salah.
- Tiga alat *debugging* — MS-DEBUG, GUI Turbo Assembler, dan Emu8086 — menampilkan informasi yang sama; MS-DEBUG dipakai sebagai acuan karena memperlihatkan keadaan mesin apa adanya.

BAGIAN

Menelusuri Program dengan Perintah DEBUG

Memuat dan Menulis Berkas: `N`, `L`, `W`

`N` (*name*) menetapkan nama berkas, `L` (*load*) memuat isi berkas ke memori, dan `W` (*write*) menulis kembali isi memori ke berkas.

```
C:\ASM>debug bug-loop.exe
-n bug-loop.exe
-l
```

- `L` *tidak* menjalankan program; ia hanya memindahkan isi berkas ke memori dan menetapkan register segmen beserta titik masuknya.
- `L` dan `W` bekerja pada berkas yang *terakhir* ditetapkan `N`, sehingga urutan `N` lalu `L` harus selalu dijaga.

Sumber: Bab XV, Memuat dan Menulis Berkas

Perintah R: Memeriksa Register dan *Flag*

R menampilkan seluruh register umum, register segmen, dan seluruh *flag* dalam satu layar. Baris kedua keluarannya juga menampilkan instruksi yang sedang ditunjuk CS:IP. R dapat pula dipakai untuk *mengubah* isi register, misalnya R AX atau R IP.

```
-r
AX=13A6  BX=0000  CX=00FF  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=1391  ES=1391  SS=13A7  CS=13A1  IP=0000   NV UP EI PL NZ NA PO NC
13A1:0000 B8A613          MOV     AX,13A6
```

Delapan penanda *flag* ditampilkan dengan urutan yang selalu tetap: OF, DF, IF, SF, ZF, AF, PF, lalu CF. Menghafal urutan ini sangat membantu ketika mengisi tabel *trace*.

Sumber: Bab XV, Memeriksa Register dengan R

Perintah U: Membongkar Kode Mesin

U (*unassemble*) menerjemahkan kembali kode mesin menjadi mnemonic assembly. Perintah ini dipakai untuk memastikan bahwa kode yang dijalankan benar-benar sama dengan yang ditulis, dan untuk mengetahui di alamat berapa setiap instruksi berada.

```
-u 0000 001c
13A1:0000 B8A613          MOV     AX,13A6
13A1:0003 8ED8            MOV     DS,AX
13A1:0005 31C0            XOR     AX,AX
13A1:0007 B90500          MOV     CX,0005
13A1:000A 01C8            ADD     AX,CX
13A1:000C 41              INC     CX
13A1:000D E2FB            LOOP    000A
13A1:000F B409            MOV     AH,09
13A1:0011 BA0000          MOV     DX,0000
13A1:0014 CD21            INT     21
13A1:0016 B44C            MOV     AH,4C
13A1:0018 B000            MOV     AL,00
13A1:001A CD21            INT     21
```

Sumber: Bab XV, Membongkar Kode Mesin dengan U

Penelusuran Satu Langkah dengan T

T (*trace*) menjalankan *tepat satu* instruksi, lalu menampilkan kembali seluruh register dan *flag*, serta mnemonic instruksi berikutnya. Inilah perintah terpenting pada bab ini: hanya dengan menjalankan satu instruksi demi satu instruksi kita dapat melihat pada langkah berapa sebuah nilai mulai menyimpang.

```
-t
AX=13A6  BX=0000  CX=00FF  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=13A6  ES=1391  SS=13A7  CS=13A1  IP=0005   NV UP EI PL NZ NA PO NC
13A1:0005 31C0           XOR     AX,AX
```

Baris terakhir selalu menunjukkan instruksi yang *akan* dijalankan berikutnya, bukan yang baru saja selesai. Pada langkah di atas `DS` sudah bernilai `13A6h` karena `MOV DS, AX` baru saja dieksekusi.

Sumber: Bab XV, Penelusuran Satu Langkah dengan T

Sesi Perulangan: CX yang Tidak Pernah Nol

Inti temuan pada bug-loop.exe: CX naik menjadi 0006h oleh INC CX, lalu LOOP menguranginya kembali menjadi 0005h.

```
-t
AX=0000  BX=0000  CX=0005  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=13A6  ES=1391  SS=13A7  CS=13A1  IP=000A   NV UP EI PL ZR NA PE NC
13A1:000A 01C8            ADD     AX,CX
-t
AX=0005  BX=0000  CX=0005  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
13A1:000C 41              INC     CX
-t
AX=0005  BX=0000  CX=0006  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
13A1:000D E2FB            LOOP    000A
-t
AX=0005  BX=0000  CX=0005  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
13A1:000A 01C8            ADD     AX,CX
```

Sumber: Bab XV, Sesi DEBUG lengkap pada bug-loop.exe

Perintah DEBUG Inti: Penelusuran

Perintah	Kegunaan	Contoh	Keluaran yang muncul
`R`	Menampilkan dan mengubah register serta *flag*	`R` atau `R AX`	`AX=13A6 ... NV UP EI PL NZ NA PO NC`
`U`	Membongkar kode mesin menjadi mnemonic	`U 0000 001C`	`13A1:0000 B8A613 MOV AX,13A6`
`T`	Menjalankan satu instruksi, termasuk masuk ke dalam `CALL`	`T`	Register baru, lalu mnemonic berikutnya
`P`	Menjalankan sampai instruksi berikutnya, melewati `CALL`/`INT`	`P`	Register baru setelah prosedur selesai
`G`	Menjalankan sampai alamat tertentu (*breakpoint*)	`G 000A`	Berhenti di `000Ah` dan menampilkan register
`D`	Menampilkan isi memori (*dump*)	`D DS:0`	`13A6:0000 4A 75 6D 6C ... Jumlah 1..5 = $.`

Sumber: Bab XV, Tabel perintah DEBUG

Perintah DEBUG Inti: Memori dan Berkas

Perintah	Kegunaan	Contoh	Keluaran yang muncul
`E`	Mengubah isi memori satu per satu	`E DS:0 48 61 6C 6F 24`	Memori `DS:0000h` berubah menjadi `Halo$`
`A`	Merakit instruksi langsung di memori	`A 0100`	`13A1:0100 MOV AH,02` (DEBUG meminta baris berikutnya)
`N`	Menetapkan nama berkas	`N latihan.com`	Tidak ada keluaran
`L`	Memuat berkas yang telah dinamai ke memori	`L`	Tidak ada keluaran; register terisi
`W`	Menulis isi memori ke berkas	`W 0100`	`Writing 0020 bytes`

`L` dan `W` selalu bekerja pada berkas yang terakhir ditetapkan `N`; urutan `N` lalu `L` tidak boleh dibalik.

Sumber: Bab XV, Tabel perintah DEBUG

Perbedaan `T` dan `P`

Keduanya sama-sama menjalankan program, tetapi berbeda dalam menyikapi instruksi yang memindahkan kendali.

`T` — *trace*

Menjalankan satu instruksi *apa pun*. Jika instruksi itu `CALL`, `T` *masuk* ke dalam prosedur; jika `INT 21h`, `T` masuk ke rutin layanan DOS.

`P` — *proceed*

Menjalankan instruksi sampai instruksi berikutnya pada level yang sama. Jika instruksi itu `CALL` atau `INT`, `P` menjalankannya sampai selesai lalu berhenti setelahnya.

Penting: Pakai `P` untuk `INT 21h` dan prosedur yang sudah diyakini benar; pakai `T` untuk prosedur yang sedang dicurigai. `P` *tidak memperbaiki* bug di dalam prosedur — ia hanya menyembunyikan langkah-langkahnya.

Sumber: Bab XV, Perbedaan T dan P

Breakpoint dengan Perintah G

Menelusuri ratusan instruksi dengan T jelas tidak efisien. Perintah G (*go*) menjalankan program sampai alamat tertentu berhenti, sehingga programmer dapat langsung melompat ke bagian yang dicurigai.

- `G 000A` menjalankan instruksi dari `0000h` sampai `000Ah` dan berhenti *sebelum* instruksi pada alamat itu dieksekusi.
- Beberapa alamat dapat ditulis sekaligus, misalnya `G 000A 000F`; program berhenti pada alamat pertama yang tercapai, sehingga titik di dalam dan sesudah perulangan dapat diperiksa dalam satu kali jalan.

Keterbatasan DEBUG klasik

(1) jumlah *breakpoint* terbatas, umumnya empat sampai sepuluh; (2) *breakpoint* hanya pada *alamat*, bukan baris sumber atau kondisi seperti "berhenti bila `AX` bernilai nol"; (3) hanya mode real 16-bit, sehingga program 32-bit tidak dapat ditelusuri.

Sumber: Bab XV, Breakpoint dengan Perintah G

Tabel Trace Register dan *Flag*

L	Alamat	Instruksi	AX	CX	SP	ZF	CF	OF
1	13A1:0000	MOV AX,13A6	13A6	0000	0100	0	0	0
2	13A1:0003	MOV DS,AX	13A6	0000	0100	0	0	0
3	13A1:0005	XOR AX,AX	0000	0000	0100	1	0	0
4	13A1:0007	MOV CX,0005	0000	0005	0100	1	0	0
5	13A1:000A	ADD AX,CX	0005	0005	0100	0	0	0
6	13A1:000C	INC CX	0005	0006	0100	0	0	0
7	13A1:000D	LOOP 000A	0005	0005	0100	0	0	0
8	13A1:000A	ADD AX,CX	000A	0005	0100	0	0	0
9	13A1:000C	INC CX	000A	0006	0100	0	0	0
10	13A1:000D	LOOP 000A	000A	0005	0100	0	0	0
11	13A1:000A	ADD AX,CX	000F	0005	0100	0	0	0

Penyebabnya tampak pada langkah 6 dan 7: `CX` berayun pada 5 dan 6 sehingga tidak pernah mencapai nol. `ZF` pada langkah 4 masih `1` karena `MOV` *tidak pernah* mengubah *flag*.

Sumber: Bab XV, Tabel trace register dan flag

Rangkuman Bagian 2

- Berkas dimuat dengan urutan `N` lalu `L`; `L` hanya memindahkan isi berkas ke memori, bukan menjalankannya.
- `R` menampilkan register dan *flag* dengan urutan tetap `OF–DF–IF–SF–ZF –AF–PF–CF`, sekaligus instruksi berikutnya; `U` membongkar kode mesin menjadi mnemonic.
- `T` menjalankan satu instruksi dan masuk ke dalam `CALL` serta `INT`; `P` melewatinya. `P` tidak memperbaiki bug, hanya menyembunyikan langkahnya.
- `G alamat` memasang *breakpoint*, tetapi DEBUG klasik hanya menyediakan titik henti dalam jumlah terbatas dan hanya pada alamat, bukan pada baris sumber atau kondisi.
- Bukti utama *debugging* adalah tabel *trace*: pada setiap langkah dicatat alamat, instruksi, register, dan *flag*.

BAGIAN

Analisis Memori dan Stack serta Metodologi Debugging

Membaca Keluaran D

D menampilkan 128 byte memori dalam delapan baris. Setiap baris memuat tiga bagian yang dibaca secara berbeda.

```
-d ds:0
13A6:0000 4A 75 6D 6C 61 68 20 31-2E 2E 35 20 3D 20 24 00  Jumlah 1..5 = $.
```

- **Alamat** dalam bentuk *segmen*:*offset*, yaitu 13A6h:0000h.
- **Isi memori**: enam belas pasang digit heksadesimal; setiap dua digit adalah satu byte.
- **Kolom karakter ASCII** di sebelah kanan; byte yang bukan karakter cetak ditampilkan sebagai titik.

Kolom karakter memperlihatkan string `Jumlah 1..5 = $`; kolom heksadesimal memperlihatkan kode ASCII-nya, 4Ah untuk J, 75h untuk u, sampai 24h untuk tanda $; byte terakhir 00h adalah penanda akhir yang dipasang assembler.

Sumber: Bab XV, Membaca Keluaran D

Memeriksa Variabel: Kesalahan *Off-by-One*

Program bermaksud menjumlahkan lima byte larik `nilai`; jumlah yang benar adalah 150 (`0096h`).

```
-d ds:0
13A6:0000  0A 14 1E 28 32 B7 00 00-00 00 00 00 00 00 00 00  ...(2...........
```

- Lima byte pertama sesuai data: `0Ah`=10 sampai `32h`=50.
- Byte keenam bernilai `B7h`, yaitu sisa data lama di memori, bukan bagian dari larik.
- Karena penghitung bernilai 6, byte itu ikut dijumlahkan sehingga akumulator menghasilkan `014Dh` (333) alih-alih `0096h`.

Bukti ini mengubah dugaan menjadi kepastian: kesalahannya bukan pada penjumlahan, melainkan pada batas perulangan.

Sumber: Bab XV, Memeriksa Variabel dengan D

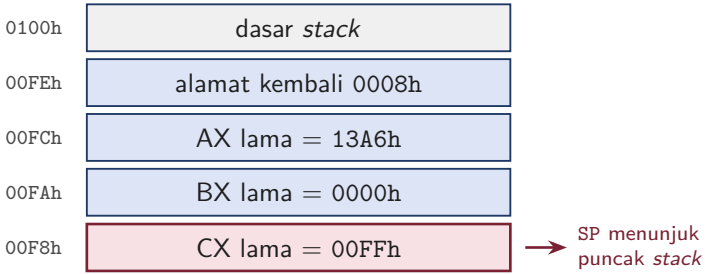
Program Larik Ber-kesalahan *Off-by-One*

```
.DATA
    nilai  DB  10, 20, 30, 40, 50   ; lima byte
.CODE
START:
    XOR  DX, DX           ; DX = akumulator jumlah
    MOV  CX, 6            ; BUG: seharusnya MOV CX, 5
    MOV  SI, 0            ; SI = indeks larik
ULANG:
    MOV  AH, 0            ; nolkan AH
    MOV  AL, nilai[SI]    ; ambil elemen ke-SI
    ADD  DX, AX           ; tambahkan ke akumulator
    INC  SI               ; naikkan indeks
    LOOP ULANG            ; ulangi sebanyak CX
```

- Larik berisi lima byte, sedangkan penghitung diisi 6, sehingga satu byte di luar larik ikut dibaca.
- Perbaikannya: ubah `MOV CX, 6` menjadi `MOV CX, 5`.
- Cara yang lebih aman adalah menghitung jumlah elemen otomatis dengan `panjang EQU $ - nilai`.

Program ditutup `MOV AH, 4Ch` dan `INT 21h`. Sumber: Bab XV, Studi Kasus Tiga Bug (Kasus 2).

Membaca Bingkai Stack



`CALL` menurunkan `SP` dari `0100h` ke `00FEh` dan menuliskan alamat kembali di sana; ketiga `PUSH` menurunkannya sampai `00F8h`. Alamat kembali berada *di bawah* seluruh data yang ditekan prosedur, sehingga `RET` hanya dapat mengambilnya bila seluruh data di atasnya sudah diambil kembali. Sumber: Bab XV, Membaca Bingkai Stack.

Menemukan Stack Tidak Seimbang

Prosedur menekan tiga register tetapi hanya memulihkan dua, sehingga RET membaca nilai yang bukan alamat kembali.

```
-g 0005
13A1:0005 E80D00          CALL    0015
-t
AX=13A6  BX=0000  CX=00FF  DX=0000  SP=00FE  BP=0000  SI=0000  DI=0000
13A1:0015 50              PUSH    AX
-t
AX=13A6  BX=0000  CX=00FF  DX=0000  SP=00FC  BP=0000  SI=0000  DI=0000
13A1:0016 53              PUSH    BX
-t
AX=13A6  BX=0000  CX=00FF  DX=0000  SP=00FA  BP=0000  SI=0000  DI=0000
13A1:0017 51              PUSH    CX
-d ss:00f8
13A7:00F8  FF 00 00 00 A6 13 08 00-00 00 00 00 00 00 00 00  ................
-t
AX=0003  BX=0002  CX=00FF  DX=0000  SP=00FC  BP=0000  SI=0000  DI=0000
13A1:0024 C3              RET
```

Setelah dua POP, SP berhenti pada 00FCh dan menunjuk A6 13, yaitu nilai AX lama; RET memuatnya ke IP sehingga program melompat ke 13A6h — jauh di luar batas program.

Sumber: Bab XV, Menemukan stack tidak seimbang melalui D dan T

Deteksi *Overflow* melalui *Flag* dan SP

Overflow aritmatika tidak selalu menghentikan program, tetapi selalu meninggalkan jejak pada *flag*.

Overflow aritmatika

Operasi yang melewati batas bilangan *bertanda* menyalakan OF, sedangkan yang melewati batas bilangan *tak bertanda* menyalakan CF.

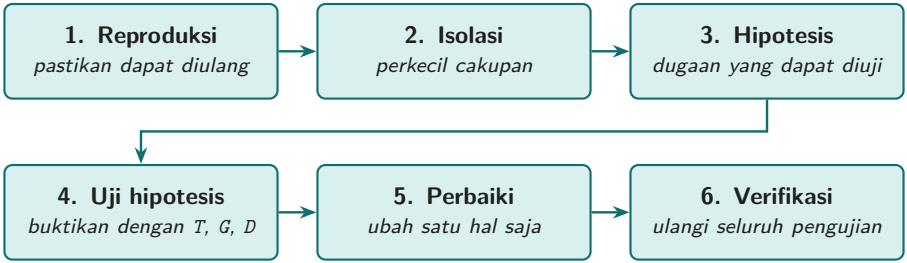
Overflow stack

Meninggalkan jejak berbeda: nilai SP terus menurun tanpa pernah naik kembali. Gejalanya adalah perubahan nilai variabel yang tidak pernah disentuh program.

Karena itu CF, OF, dan register SP sebaiknya selalu disertakan pada tabel *trace*.

Sumber: Bab XV, Deteksi Overflow melalui Flag dan SP

Metodologi Debugging yang Sistematis



Pada tahap perbaikan, ubah *satu hal saja* agar penyebab keberhasilan dapat dipastikan. Bila program kemudian berjalan benar, mengubah beberapa baris sekaligus membuat kita tidak pernah tahu baris mana yang sebenarnya memperbaiki keadaan.

Sumber: Bab XV, Metodologi Debugging yang Sistematis

Studi Kasus Tiga Bug

1. **Perulangan tak berujung.** `bug-loop.asm` berhenti tidak pada waktunya karena `CX` dinaikkan di dalam badan perulangan; `LOOP` menguranginya kembali, sehingga `CX` berayun pada 5 dan 6. Perbaikannya menghapus `INC CX`.
2. **Kesalahan *off-by-one* pada larik.** Program selesai normal, sehingga kesalahan sintaks dan kesalahan saat berjalan dapat disingkirkan. `D DS:0` memperlihatkan byte keenam di luar larik; perbaikannya mengubah `MOV CX, 6` menjadi `MOV CX, 5`.
3. ***Stack* tidak seimbang.** `bug-stack.asm` melompat ke alamat aneh — gejala khas *stack* rusak, karena hanya `CALL`, `RET`, dan `INT` yang dapat memindahkan kendali ke luar batas program. Prosedur menekan tiga register tetapi memulihkan dua; perbaikannya menambahkan satu `POP AX`.

Pola yang sama pada ketiganya: gejalanya muncul di satu tempat, sedangkan penyebabnya berada di tempat lain.

Sumber: Bab XV, Studi Kasus Tiga Bug

Rangkuman Bagian 3

- Keluaran `D` dibaca dalam tiga bagian: alamat, isi memori heksadesimal, dan kolom karakter ASCII. Kolom karakter paling cepat dipakai untuk mengenali letak sebuah string.
- Bagian *stack* dibaca dari alamat terendah ke tertinggi: data yang ditekan terakhir berada pada alamat terendah, sedangkan alamat kembali berada di bawah seluruh data tersebut.
- *Overflow* aritmatika meninggalkan jejak pada `OF` (bilangan bertanda) dan `CF` (bilangan tak bertanda); *overflow stack* meninggalkan `SP` yang terus menurun.
- Metodologi *debugging* terdiri atas enam tahap: reproduksi, isolasi, hipotesis, pengujian, perbaikan, dan verifikasi — dengan perbaikan satu hal pada satu waktu.
- Tiga bug yang dibedah adalah perulangan tak berujung, kesalahan *off-by-one* pada larik, dan *stack* tidak seimbang.

Contoh 1: Perulangan Tak Berujung

Bermaksud menjumlahkan bilangan 1 sampai 5 (seharusnya 000Fh atau 15); program dirakit tanpa kesalahan sintaks, tetapi tidak pernah selesai.

bug-loop.asm (ber-bug)

```
XOR AX, AX       ; akumulator = 0
MOV CX, 5        ; penghitung = 5
ULANG:
    ADD AX, CX   ; jumlahkan penghitung
    INC CX       ; BUG: CX dinaikkan
    LOOP ULANG   ; LOOP mengurangi CX
MOV AH, 09h      ; tak pernah dicapai
LEA DX, pesan
INT 21h
```

bug-loop-fix.asm (perbaikan)

```
XOR AX, AX       ; akumulator = 0
MOV CX, 5        ; penghitung = 5
ULANG:
    ADD AX, CX
    LOOP ULANG   ; CX turun otomatis
MOV BX, AX       ; BX = 15
MOV AH, 09h      ; cetak judul
LEA DX, pesan
INT 21h
```

Keluaran

```
Jumlah 1..5 = 15
```

Contoh 2: Stack yang Tidak Seimbang

Program memanggil prosedur `HITUNG` lalu bermaksud mencetak pesan penutup; kendali melompat ke alamat yang tidak dikenal dan pesan itu tidak pernah muncul.

`bug-stack.asm` (ber-bug)

```
HITUNG PROC
    PUSH AX      ; simpan AX lama
    PUSH BX      ; simpan BX lama
    PUSH CX      ; simpan CX lama
    MOV  AX, 0001h
    MOV  BX, 0002h
    ADD  AX, BX ; AX = 0003h
    MOV  CX, AX
    POP  CX      ; pulihkan CX
    POP  BX      ; pulihkan BX
    RET          ; BUG: POP AX tidak ada
HITUNG ENDP
```

`bug-stack-fix.asm` (perbaikan)

```
    MOV  CX, AX
    POP  CX      ; pulihkan CX
    POP  BX      ; pulihkan BX
    POP  AX      ; PERBAIKAN: pulihkan AX
    RET          ; alamat kembali benar
HITUNG ENDP
```

Keluaran

```
Selesai.
```

Setiap `PUSH` memerlukan `POP` pasangannya pada jalur eksekusi yang sama.

Praktikum: Debugging Terpandu Tiga Kasus Bug

Judul praktikum

Debugging terpandu tiga kasus bug tingkat instruksi ▪ **Sub-CPMK:** 4.1 dan 4.2 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu, laporan tertulis

Alat dan bahan

- Komputer bersistem operasi Windows
- DOSBox beserta berkas `DEBUG.EXE`
- TASM dan TLINK (atau Emu8086 sebagai pembanding)
- Berkas `bug-loop.asm`, `bug-stack.asm`, dan `program-larik.asm`

Langkah kerja

1. Rakit ketiga berkas uji dengan TASM dan TLINK.
2. **Kasus 1:** muat `bug-loop.exe`, catat `R` dan `U`, lalu jalankan `T` sepuluh kali.
3. **Kasus 2:** jalankan `G` sampai setelah `LOOP`, lalu `D DS:0` untuk memeriksa larik.
4. **Kasus 3:** `G 0005`, telusuri dengan `T`, lalu bandingkan `T` dan `P` pada `CALL`.
5. Tuliskan hipotesis, perbaiki, rakit ulang, dan verifikasi.

Dikumpulkan: tabel *trace* lengkap (minimal sepuluh langkah per kasus) dengan kolom `AX`, `BX`, `CX`, `DX`, `SP`, `ZF`, `CF`, `OF`; salinan keluaran `R`, `U`, `D`, dan `T`; berkas hasil perbaikan beserta tangkapan layarnya.

Latihan

1. Jelaskan perbedaan lima jenis kesalahan program. Untuk setiap jenis, sebutkan satu contoh dan alat atau perintah yang paling tepat untuk menemukannya.

2. Sebuah program memanggil `INT 21h` dengan fungsi `09h`. Apa yang terjadi bila ditelusuri dengan `T`, dan apa yang terjadi bila ditelusuri dengan `P`? Mengapa `P` lebih dianjurkan pada kasus ini?

3. Diberikan keluaran `R`: `AX=0000 CX=0005 SP=00F8 IP=000A` dan baris *flag* `NV UP EI NG NZ AC PO CY`. Nilai `ZF`, `CF`, `SF`, dan `OF` berturut-turut berapa? Instruksi apa yang dijalankan berikutnya?

4. Diberikan keluaran `D DS:0000`: `48 41 4C 4F 24 0A 0B ....` String apa yang tersimpan di sana, dan pada byte berapa penanda akhirnya? Apa yang tampak di layar bila program membaca delapan byte mulai `0000h`?

5. Tulislah prosedur `HITUNG` yang menekan tiga register ke *stack*, melakukan perhitungan sederhana, lalu memulihkan ketiganya sebelum `RET`. Jelaskan mengapa urutan `POP` harus kebalikan urutan `PUSH`.

Rangkuman

- *Debugging* adalah proses sistematis untuk menemukan penyebab kesalahan melalui bukti pengamatan pada level instruksi, bukan melalui perubahan kode secara acak.
- Lima jenis kesalahan program: sintaks, saat berjalan, logika, segmentasi, dan *overflow* (aritmatika maupun *stack*). Hanya kesalahan sintaks yang dapat dideteksi tanpa *debugger*.
- Tiga alat *debugging* — MS-DEBUG di DOSBox, *debugger* terpadu GUI Turbo Assembler, dan jendela *debug* Emu8086 — menampilkan informasi yang sama: register, memori, *flag*, dan *stack*.
- Perintah wajib DEBUG adalah `R`, `U`, `T`, `P`, `G`, `D`, `E`, `A`, serta `N`, `L`, dan `W`. Perintah `T` masuk ke dalam prosedur dan `INT`, sedangkan `P` melewatinya.
- Bukti utama *debugging* adalah tabel *trace*. Keluaran `D` dibaca dalam tiga bagian: alamat, isi heksadesimal, dan kolom karakter ASCII.
- Metodologi *debugging* terdiri atas enam tahap: reproduksi, isolasi, hipotesis, pengujian, perbaikan, dan verifikasi — dengan perbaikan satu hal pada satu waktu, lalu pengujian diulang pada seluruh kasus.

Cek Pemahaman

1. Perintah DEBUG yang menampilkan seluruh register dan *flag* adalah ...

   1.1 `U`
   1.2 `D`
   1.3 `R`
   1.4 `T`

2. Pada `CALL HITUNG`, perintah yang *tidak* masuk ke dalam prosedur adalah ...

   2.1 `T`
   2.2 `P`
   2.3 `U`
   2.4 `R`

3. Prosedur menekan tiga register tetapi hanya memulihkan dua di antaranya. Akibatnya adalah ...

   3.1 assembler menolak program karena `PUSH` dan `POP` tidak seimbang
   3.2 `RET` melompat ke alamat yang bukan alamat kembali
   3.3 register `AX` dipulihkan otomatis oleh DOS
   3.4 program tetap benar karena `AX` tidak dipakai lagi

Penutup

Sebelum pertemuan berikutnya

- Selesaikan laporan praktikum: tabel *trace* ketiga kasus bug, salinan keluaran `R`, `U`, `D`, dan `T`, serta berkas hasil perbaikan.
- Ulangi bagian yang belum dikuasai dengan menelusuri `bug-loop.asm` dan `bug-stack.asm` satu instruksi demi satu, lalu bandingkan keluaran `T` dan `P`.
- Baca **Bab XVI** — Analisis Kode Mesin, Disassembly, dan Optimasi.

Bacaan lanjutan: Bab XV bagian Perbedaan `T` dan `P`, Membaca Bingkai Stack, dan Metodologi *Debugging* yang Sistematis; pembahasan register pada Bab VII; pembahasan *flag* pada Bab IX; pembahasan prosedur dan *stack* pada Bab XIII.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil